Localization for Autonomous Navigation

Chapter 3. Probabilities, Gaussians, and Bayes' Theorem

Dept. Intelligent Systems & Robots

Prof. SungTae Moon



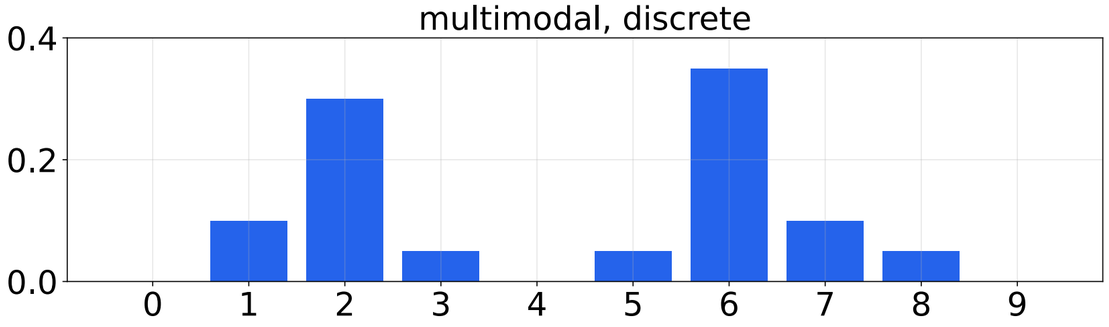
Contents

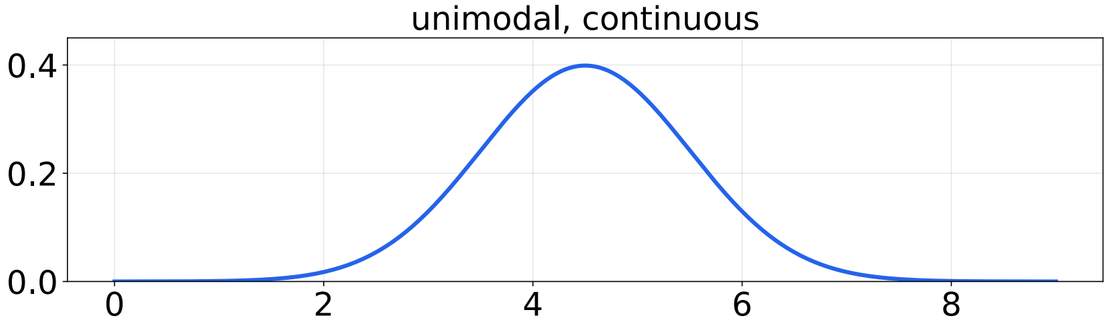
- Introduction
- Mean, Variance, and Standard Deviations
- Probability Distribution
- Expected Value of a Random Variable
- Gaussians
- Gaussian Distributions
- The Variance and Belief
- The 68-95-99.7 Rule
- Interactive Gaussians
- Computational Properties of Gaussians
- Putting it all Together
- Computing Probabilities with scipy.stats
- Limitations of Using Gaussians to Model the World
- Product of Gaussians
- Sum of Gaussians
- Summary and Key Points

Unimodal and Continuous

- **이산 표현의 한계**: 여러 위치에 동시에 확률, 이산 칸, 큰 계산량
- **원하는 표현**: 하나의 값(unimodal), 실수 연속값(continuous), 계산이 쉬운 형태

> **핵심**: 항공기는 "여기 아니면 저기" 가 아닌 하나의 위치로 표현해야 함, Gaussian 이 세 조건을 모두 만족





Random Variables

Discrete random variable

- **주사위**: 1 ~ 6, 각 1/6
- **동전**: {H, T}
- **표본 공간**: 가능한 값의 집합, 유한하거나 셀 수 있음

Continuous random variable

- **키**: 두 생물학적 한계 사이의 모든 실수
- **측정과 구분**: 0.1 m 단위로 측정해도 키 자체는 연속

> **random**: 비결정적이라는 뜻이 아니라 결과를 계산할 정보가 부족하다는 뜻, 확률 변수는 대문자 X, Y

Probability Distribution

- 위치 추정에 쓰던 이산 belief 배열을 전체 합으로 나눠 확률 분포로 만드는 예
- 각 칸의 확률은 0 ~ 1 사이이고 출력된 전체 합은 1.0
- 두 조건을 만족하므로 이산 확률 분포, 보통은 이산과 연속을 구분하지 않고 확률 분포라 부름
- **조건**: 11 칸 위치 belief [1, 4, 2, 0, 8, 2, 2, 35, 4, 3, 2] 를 전체 합으로 정규화

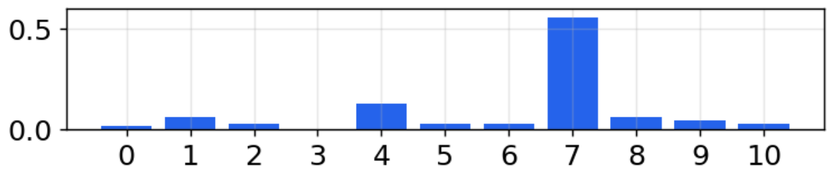
$$P(X=4) = p(4) = \frac{1}{6} \qquad \sum_u P(X=u) = 1 \qquad \int_u P(X=u)\,du = 1$$

- **표기**: 대문자 P 는 사건 하나의 확률, 소문자 p 는 확률 분포 함수

```python
import numpy as np

belief = np.array([1, 4, 2, 0, 8, 2, 2, 35, 4, 3, 2])
belief = belief / np.sum(belief)
plots.bar_plot(belief, ylim=(0, .6))
print('sum = ', np.sum(belief))
```

```
sum =  1.0
```



Mean, Median, and Mode

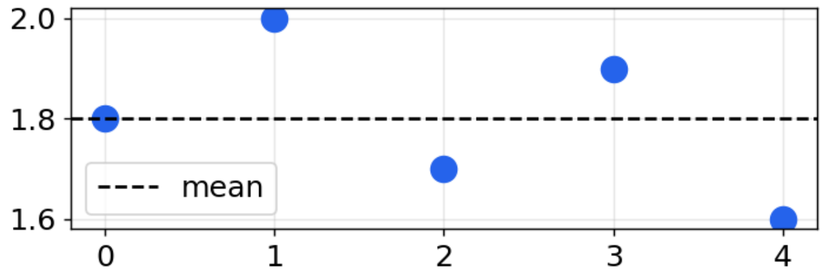
- 학생 5 명의 키로 대표값인 평균(합 / 개수)과 중앙값(정렬 후 가운데 값)을 계산
- 결과는 평균과 중앙값 모두 1.8 m, 일반적으로는 두 값이 같지 않음
- 최빈값 mode 는 가장 자주 나오는 값, 하나면 unimodal, 둘 이상이면 multimodal
- **조건**: 학생 5 명 키 1.8, 2.0, 1.7, 1.9, 1.6 m

$$\mu = \frac{1.8+2.0+1.7+1.9+1.6}{5} = 1.8 \qquad \mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

```python
x = np.array([1.8, 2.0, 1.7, 1.9, 1.6])
print(np.mean(x), x.mean(), np.median(x))

plt.plot(x, 'o', ms=12)
plt.axhline(np.mean(x), ls='--', color='k', label='mean')
plt.legend()
```

```
1.8 1.8 1.8
```



Expected Value

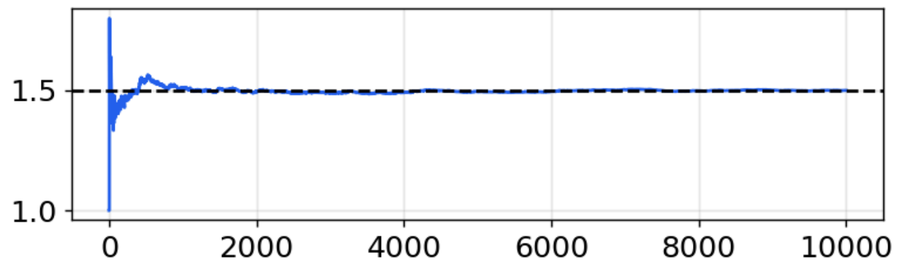
- 값 1, 3, 5 가 서로 다른 확률로 나오는 확률 변수를 난수로 10000 번 뽑아 평균
- 누적 평균이 처음에는 흔들리다가 해석으로 구한 기댓값 1.5 에 가까워짐
- 기댓값은 무한히 많은 표본의 평균, 유한 표본이라 정확히 1.5 는 아님
- **조건**: P(1) = 0.8, P(3) = 0.15, P(5) = 0.05, 표본 10000 개

$$E[X] = (1)(0.8) + (3)(0.15) + (5)(0.05) = 1.5 \qquad E[X] = \sum_{i=1}^{n} p_i x_i \qquad E[X] = \int_a^b x f(x)\, dx$$

```python
N = 10000
samples = []
for r in np.random.rand(N):
    if r <= .80: samples.append(1)
    elif r < .95: samples.append(3)
    else: samples.append(5)

running = np.cumsum(samples) / np.arange(1, N + 1)
print('expected value ~', running[-1])
plt.plot(running)
plt.axhline(1.5, ls='--', color='k')
```

```
expected value ~ 1.5
```



Expected Value of Die and Uniform

- 기댓값 정의를 이산 분포(주사위)와 연속 분포(구간 [0, 20] 균등 분포)에 적용
- 각 1000 개 표본의 평균이 해석값 3.5 와 10 에 가까운지 확인
- **조건**: 주사위 1 ~ 6 각 1/6, 균등 분포 f(x) = 1/20, 각 1000 샘플

Die roll

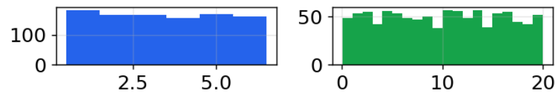
$$E[X] = \frac{1}{6}(1+2+3+4+5+6) = 3.5$$

Uniform on [0, 20]

$$E[X] = \int_0^{20} x \frac{1}{20} dx = \left[\frac{x^2}{40}\right]_0^{20} = 10$$

```
1  die = np.random.randint(1, 7, size=1000)
2  u = np.random.uniform(0, 20, size=1000)
3  print('die mean     ', die.mean())
4  print('uniform mean ', u.mean())
5
6  plt.subplot(121); plt.hist(die, bins=np.arange(.5, 7));
7  plt.subplot(122); plt.hist(u, bins=20, color='C2');
```

```
die mean      3.444
uniform mean  9.91911979115591
```



Variance

- 세 반 X, Y, Z 학생 5 명의 키로 평균과 분산을 계산하고 반별로 점을 찍어 비교
- 세 반 평균은 모두 1.8 m 이지만 Y 는 크게 퍼지고 Z 는 전혀 퍼지지 않음
- 평균만으로는 데이터를 다 설명하지 못하므로 퍼짐을 나타내는 분산이 필요
- **조건**: X = [1.8, 2.0, 1.7, 1.9, 1.6], Y 는 1.3 ~ 2.3 m, Z 는 모두 1.8 m

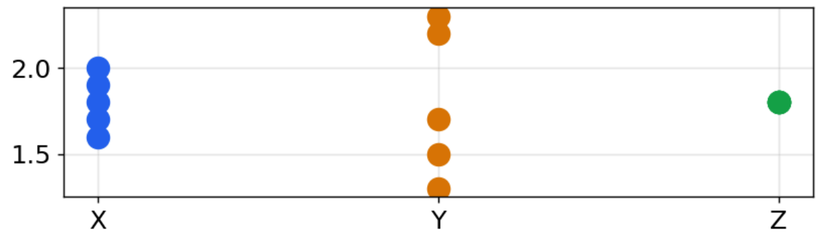
$$VAR(X) = E[(X-\mu)^2] = \frac{1}{n}\sum_{i=1}^{n}(x_i-\mu)^2 \qquad VAR(X) = \frac{0+0.04+0.01+0.01+0.04}{5} = 0.02\ m^2$$

```python
X = [1.8, 2.0, 1.7, 1.9, 1.6]
Y = [2.2, 1.5, 2.3, 1.7, 1.3]
Z = [1.8, 1.8, 1.8, 1.8, 1.8]

print(np.mean(X), np.mean(Y), np.mean(Z))
print("{:.2f} meters squared".format(np.var(X)))

for k, cls in enumerate((X, Y, Z)):
    plt.plot([k] * 5, cls, 'o', ms=12)
plt.xticks([0, 1, 2], ['X', 'Y', 'Z']);
```

```
1.8 1.8 1.8
0.02 meters squared
```



Standard Deviation

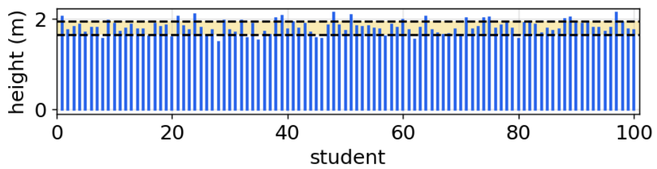
- 평균 1.8 m, 표준편차 0.1414 m 로 학생 100 명의 키를 만들어 막대와 ±1 표준편차 띠로 표시
- 띠 안에 든 학생 비율은 약 68%, 5 명일 때의 60% 보다 68% 에 가까움
- 표준편차는 분산의 제곱근, 데이터와 같은 단위(m)라 해석이 쉬움
- **조건**: 키 = 1.8 + 0.1414 randn(100), 계산된 평균과 표준편차 기준 ±1 구간

$$\sigma = \sqrt{VAR(X)} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2} \qquad \sigma_x = 0.1414$$

```python
from numpy.random import randn
data = 1.8 + randn(100)*.1414
mean, std = data.mean(), data.std()

plot_height_std(data, lw=2)
print('mean = {:.3f}'.format(mean))
print('std = {:.3f}'.format(std))
inside = (data > mean-std) & (data < mean+std)
print(np.sum(inside) / len(data) * 100.)
```

```
mean = 1.802
std = 0.144
67.0
```





Why the Square

- **편차의 합**: 항상 0 이라 부호를 없애야 퍼짐을 잴 수 있음, 방법은 절댓값 또는 제곱
- **제곱의 장점**: 큰 편차를 더 크게 반영, 절댓값으로는 같은 X 와 Y 를 분산 9 와 12.25 로 구분
- **제곱의 대가**: 이상값 하나에 분산이 크게 흔들림, 100 하나가 분산의 93% 를 차지해 2.03 에서 621.45 로 증가

Absolute Value vs Square

`X = [3, -3, 3, -3]   Y = [6, -2, -3, 1]`

	X	Y
평균	0	0.5
편차	[3, -3, 3, -3]	[5.5, -2.5, -3.5, 0.5]
편차의 합	0	0
절댓값 평균	**3**	**3** (구분 불가)
분산 (제곱 평균)	**9**	**12.25** (더 큼)
표준편차	3	3.5 (책의 "variance 3.5")

Effect of One Outlier

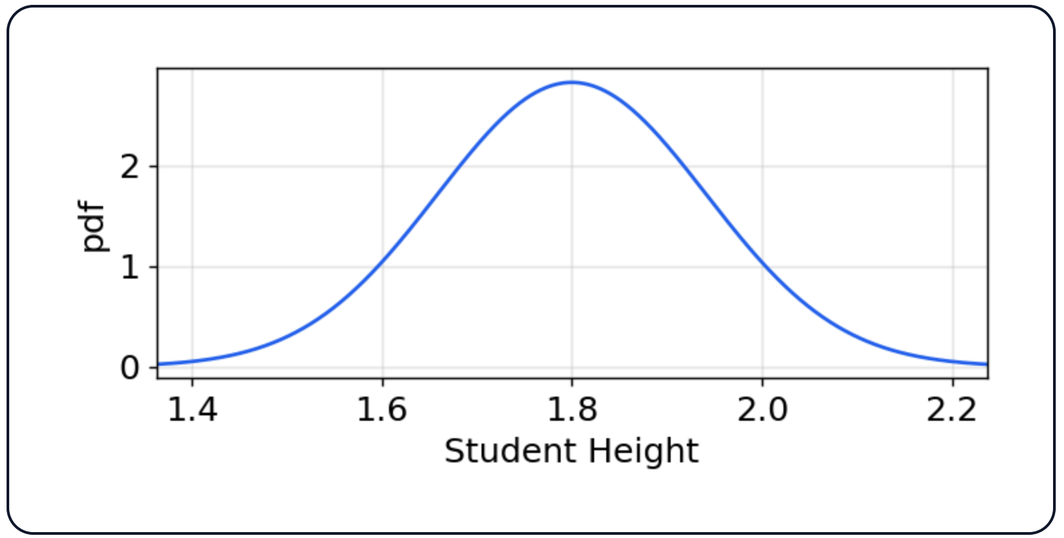
`Z = [1, -1, 1, -2, -1, 2, 1, 2, -1, 1, -1, 2, 1, -2, 100]`

	Z[:-1] (100 제외)	Z (100 포함)
평균	0.21	6.87
분산	**2.03**	**621.45**
표준편차	1.42	24.93
절댓값 평균	1.33	12.42
중앙값	1	1
100 의 비중	-	분산 93%, 절댓값 50%

Gaussians

- 학생 키 분포를 Gaussian 확률 밀도 함수(pdf)로 그림, pdf 는 각 값이 나올 상대적 가능성
- 1.8 m 근처 학생은 많고 1.4 m, 2.2 m 학생은 매우 적음, 곡선 중심은 평균
- 센서 잡음 등 많은 자연 현상이 이런 분포, unimodal 이고 연속이며 계산이 쉬움
- **조건**: 평균 1.8 m, 표준편차 0.1414 m (분산 0.1414^2)

```
from filterpy.stats import plot_gaussian_pdf
plot_gaussian_pdf(mean=1.8, variance=0.1414**2,
                  xlabel='Student Height', ylabel='pdf');
```

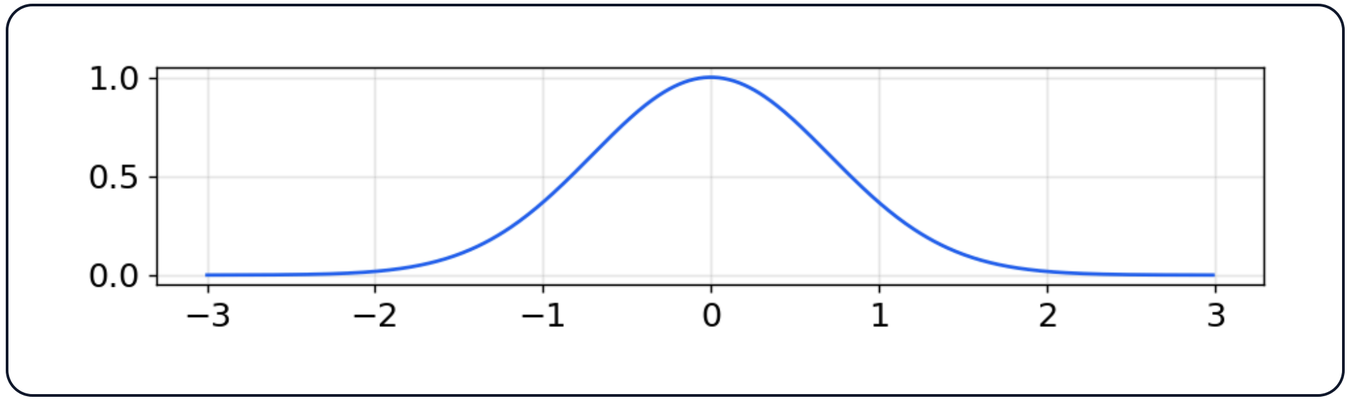


The Gaussian Equation

- Gaussian 은 평균 μ 와 분산 σ^2 두 매개변수로 완전히 결정되는 연속 확률 분포
- 식에서 상수를 빼면 단순한 지수 함수 e^{-x^2} 만 남음
- 구간 $-3 \leq x < 3$ 에서 그리면 익숙한 종 모양 곡선이 나타남
- **조건**: $-3 \leq x < 3$, 간격 $\Delta x = 0.01$, 상수 계수와 μ, σ 생략

$$f(x, \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right] \qquad f(x) \propto e^{-x^2}$$

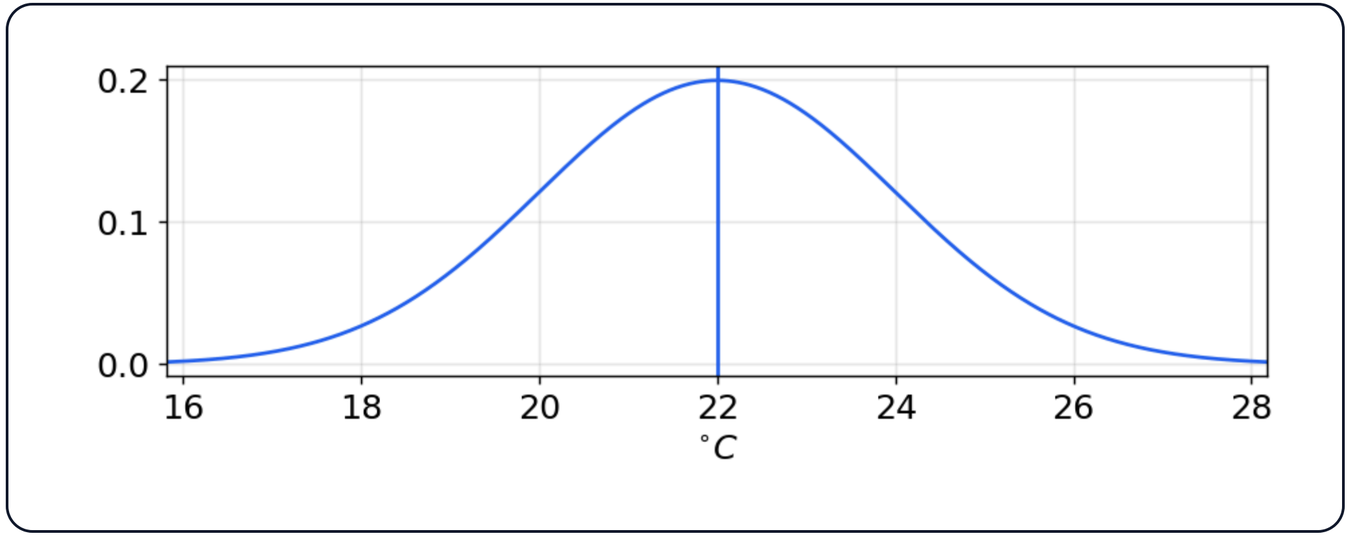
```
x = np.arange(-3, 3, .01)
plt.plot(x, np.exp(-x**2));
```



Thermometer Reading

- 22 도를 읽은 온도계의 측정값 분포를 평균 22, 분산 4 인 Gaussian 으로 그림
- 곡선은 실제 온도가 22 도일 때 온도계가 각 값을 읽을 확률에 비례, 평균에서 가장 높음
- 측정을 많이 반복하면 측정값은 정규분포를 따름 (중심 극한 정리)
- **조건**: N(22, 4): 평균 22 도, 분산 4, 표준편차 2 도, 평균선 표시

```
plot_gaussian_pdf(22, 4, mean_line=True,
                  xlabel='$^{\circ}C$');
```



Probability Density

- 온도 분포 N(22, 4) 에서 폭 1 도인 두 구간의 확률을 누적 분포(cdf)로 계산
- 평균 근처 21.5 ~ 22.5 도는 19.74%, 평균에서 먼 23.5 ~ 24.5 도는 12.10%
- 연속 분포에서 한 점의 확률은 0, 구간 확률은 곡선 아래 넓이이므로 pdf 를 적분
- **조건**: 온도 N(22, 4), 센서의 22 도 측정은 22 ± 0.1 처럼 범위를 뜻함

$$\int_{x_0}^{x_1} \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}(x-\mu)^2/\sigma^2} dx$$

```python
from filterpy.stats import norm_cdf
print('Cumulative probability of range 21.5 to 22.5 is {:.2f}%'.format(
      norm_cdf((21.5, 22.5), 22,4)*100))
print('Cumulative probability of range 23.5 to 24.5 is {:.2f}%'.format(
      norm_cdf((23.5, 24.5), 22,4)*100))
```

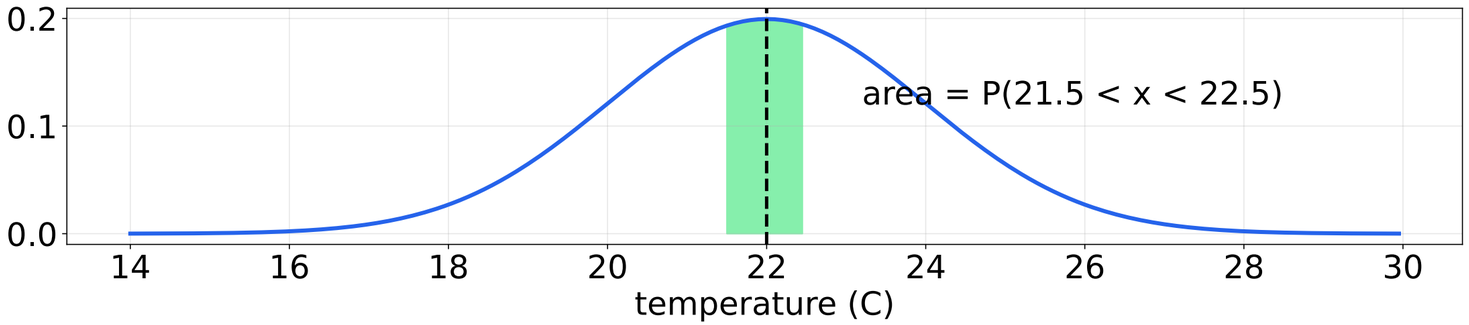
```
Cumulative probability of
range 21.5 to 22.5 is 19.74%
Cumulative probability of
range 23.5 to 24.5 is 12.10%
```

Notation

$$X \sim \mathcal{N}(\mu, \sigma^2) \qquad \text{temp} \sim \mathcal{N}(22, 4)$$

- **읽는 법**: ~ 는 "그 분포를 따른다"
- **관례**: 여기서는 항상 N(mu, sigma^2), sigma = 2, sigma^2 = 4
- **두 숫자**: 평균과 분산만으로 무한히 많은 값의 분포 표현
- **주의**: 일부 문헌은 N(mu, sigma) 표기

> **핵심**: Gaussian 은 두 숫자만으로 모든 구간의 확률을 계산할 수 있게 해 줌



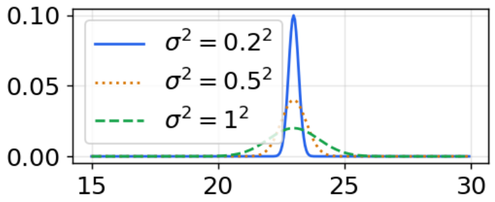
Variance and Belief

- 전체 넓이가 1 인지 확인한 뒤 평균 23 도에 분산이 다른 Gaussian 세 개를 겹쳐 그림
- 넓이 1 을 유지하므로 분산이 작으면 좁고 높게, 크면 넓고 낮게 그려짐
- 0.2^2 온도계는 23 도를 강하게 믿고, 1^2 온도계는 20 도나 26 도도 가능하다고 봄
- **조건**: 평균 23 도, 분산 0.2^2, 0.5^2, 1^2 인 세 온도계

```python
from filterpy.stats import gaussian
print(norm_cdf((-1e8, 1e8), mu=0, var=4))
print(gaussian(x=[3.0, 2.0], mean=2.0, var=1, normed=False))

xs = np.arange(15, 30, 0.05)
plt.plot(xs, gaussian(xs, 23, 0.2**2), label='$\sigma^2=0.2^2$')
plt.plot(xs, gaussian(xs, 23, .5**2), label='$\sigma^2=0.5^2$', ls=':')
plt.plot(xs, gaussian(xs, 23, 1**2), label='$\sigma^2=1^2$', ls='--')
plt.legend();
```

```
1.0
[0.242 0.399]
```

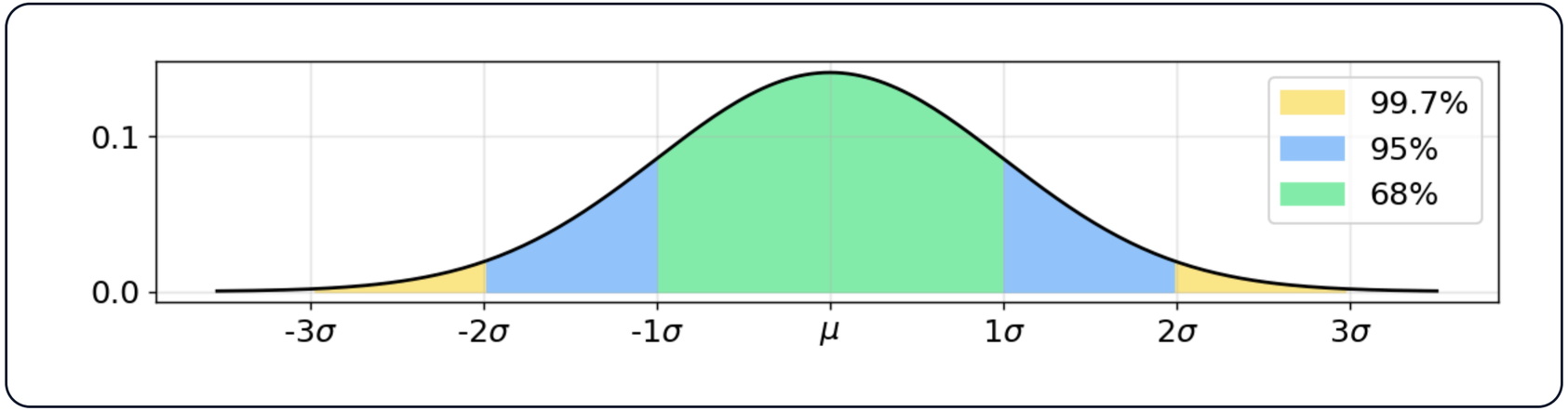


> **precision**: 분산의 역수 1/sigma^2, 분산이 작을수록 측정이 정밀하고 믿음이 강함

The 68-95-99.7 Rule

- 평균 20, 분산 8 인 Gaussian 에서 ±1, 2, 3 표준편차 구간을 색으로 구분해 그림
- 각 구간에 데이터의 68%, 95%, 99.7% 가 포함, 3 표준편차 밖은 거의 없음
- 예: 평균 71, 표준편차 9.4 인 시험이면 학생 95% 가 52.2 ~ 89.8 점
- **조건**: 평균 20, 분산 8, 표준편차 약 2.83, x 축 눈금은 표준편차 단위
- **단위**: sigma 0.2 m 면 68% 가 ± 0.2 m
- **표기**: sigma^2 = 0.2^2 로 단위 노출

```
1 display_stddev_plot()
```



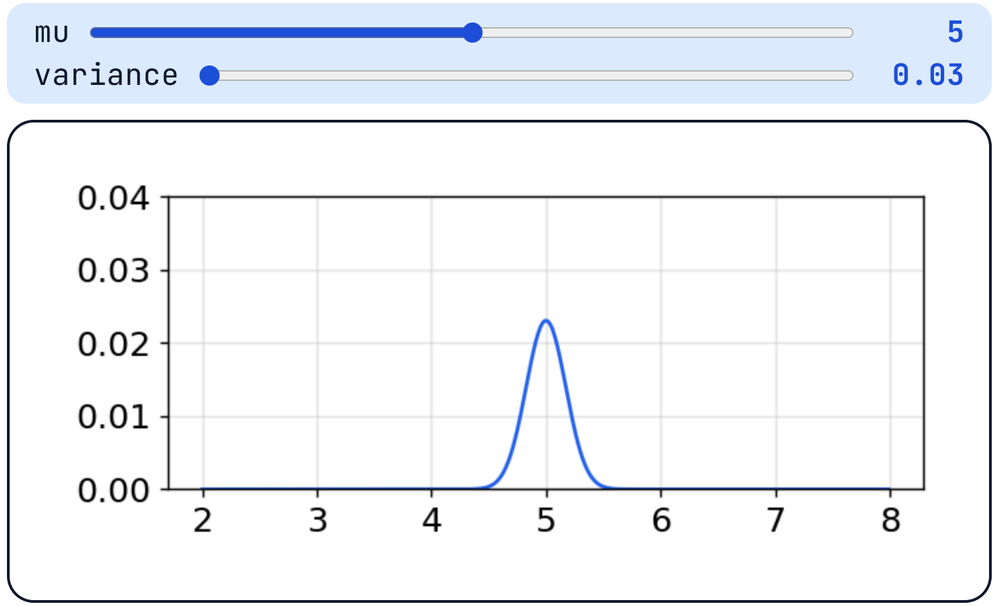
Interactive Gaussians

- 평균 mu 와 분산 variance 를 슬라이더로 바꾸며 Gaussian 곡선을 다시 그림
- mu 를 바꾸면 곡선이 좌우로 이동, variance 를 키우면 넓고 낮아지고 줄이면 좁고 높아짐
- **조건**: x 구간 2 ~ 8, mu 3 ~ 7, variance 0.01 ~ 1 슬라이더

```python
import math
from ipywidgets import interact, FloatSlider

def plt_g(mu,variance):
    plt.figure()
    xs = np.arange(2, 8, 0.01)
    ys = gaussian(xs, mu, variance)
    plt.plot(xs, ys)
    plt.ylim(0, 0.04)

interact(plt_g, mu=FloatSlider(value=5, min=3, max=7),
         variance=FloatSlider(value = .03, min=.01, max=1.));
```



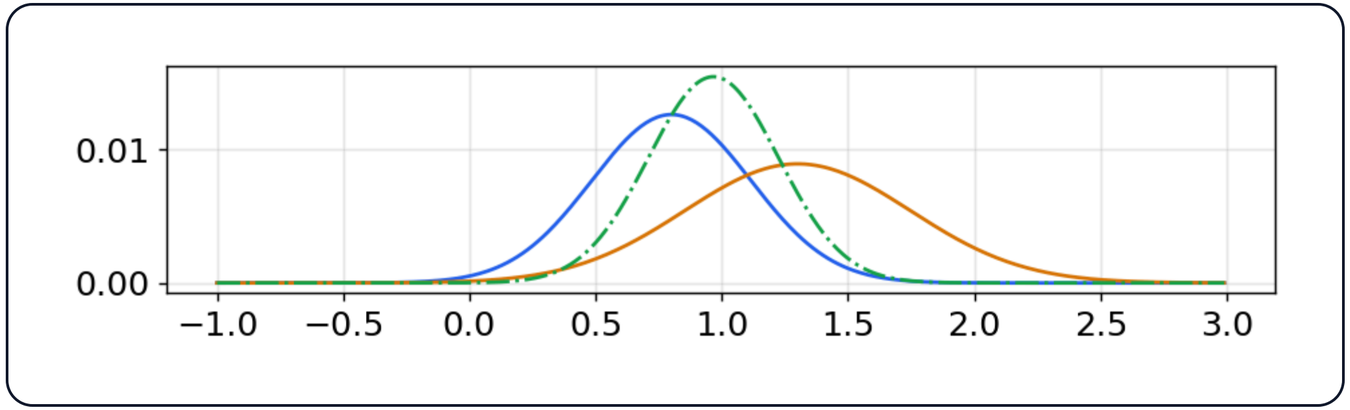
Product of Gaussians

- 두 Gaussian g1, g2 를 같은 x 위에서 원소별로 곱하고 합으로 나눠 정규화
- 곱한 결과(점선)도 Gaussian 모양, 평균은 두 평균 사이에 위치
- 합은 Gaussian, 곱은 Gaussian 에 비례, Kalman filter 가 계산 가능한 핵심 성질
- **조건**: g1 = N(0.8, 0.1), g2 = N(1.3, 0.2), x = -1 ~ 3

$$\mu = \frac{\sigma_1^2\mu_2 + \sigma_2^2\mu_1}{\sigma_1^2 + \sigma_2^2} \qquad \sigma^2 = \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2} \qquad \mu = \mu_1 + \mu_2 \qquad \sigma^2 = \sigma_1^2 + \sigma_2^2$$

```
x = np.arange(-1, 3, 0.01)
g1 = gaussian(x, mean=0.8, var=.1)
g2 = gaussian(x, mean=1.3, var=.2)
plt.plot(x, g1, x, g2)

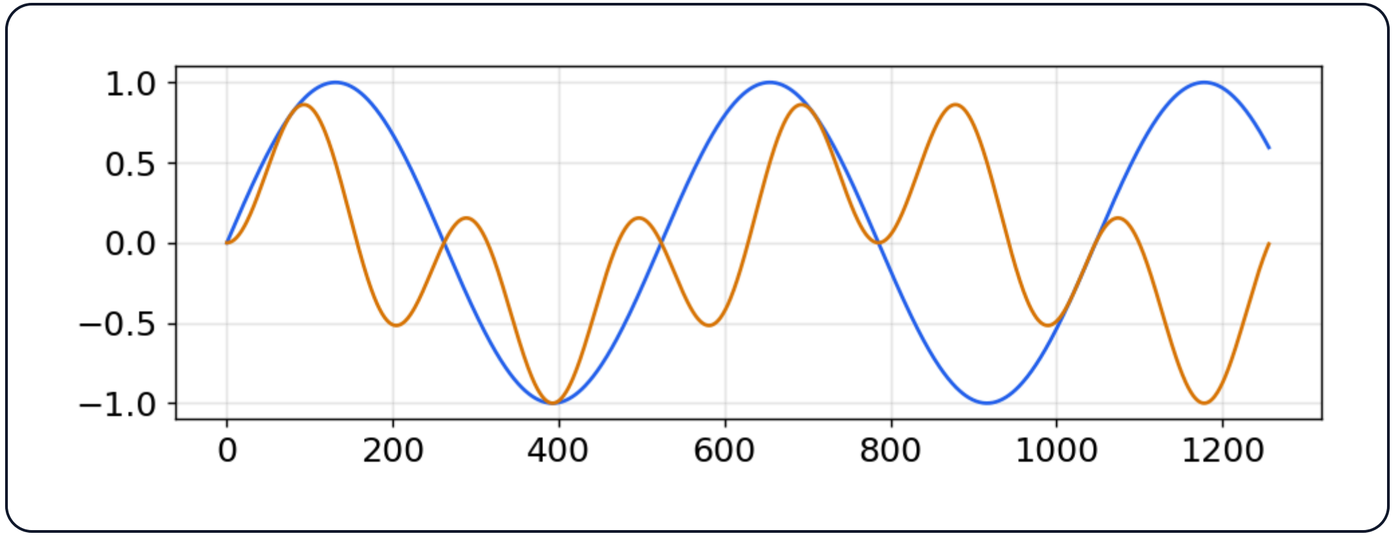
g = g1 * g2 # element-wise multiplication
g = g / sum(g) # normalize
plt.plot(x, g, ls='-.');
```



Nonlinear Functions

- Gaussian 도 비선형 함수, 다른 비선형 함수인 sin 두 개를 곱한 모양과 비교
- sin(1.2x) 와 sin(2x) 의 곱은 원래 sin 과 전혀 다른 모양
- 반면 두 Gaussian 의 곱은 다시 Gaussian 함수, Kalman filter 가 Gaussian 을 쓰는 이유
- **조건**: 0 ~ 4 pi 구간, 간격 0.01

```
x = np.arange(0, 4*np.pi, 0.01)
plt.plot(np.sin(1.2*x))
plt.plot(np.sin(1.2*x) * np.sin(2*x));
```



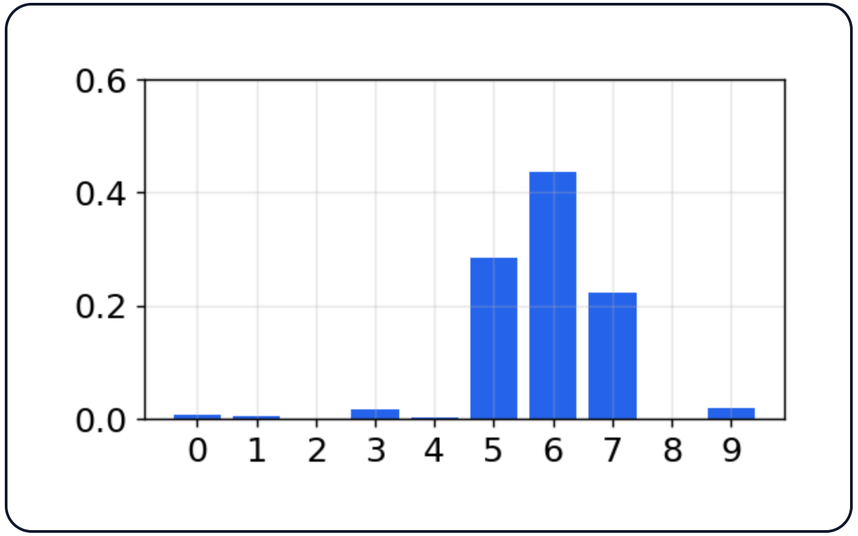
Discrete Update Revisited

- 이산 필터의 update: 각 칸의 prior 와 likelihood 를 원소별로 곱한 뒤 정규화
- 결과 posterior 는 한 위치 근처에 몰린 분포, 10 칸이므로 곱셈 10 번
- 다차원의 큰 배열이면 수십억 번의 곱셈과 많은 메모리가 필요
- **조건**: 10 칸 prior, likelihood (정수 빈도를 전체 합으로 정규화)

```
def normalize(p):
    return p / sum(p)

def update(likelihood, prior):
    return normalize(likelihood * prior)

prior =      normalize(np.array([4, 2, 0, 7, 2, 12, 35, 20, 3, 2]))
likelihood = normalize(np.array([3, 4, 1, 4, 2, 38, 20, 18, 1, 16]))
posterior = update(likelihood, prior)
plots.bar_plot(posterior, ylim=(0, .6))
```



A Gaussian Instead of an Array

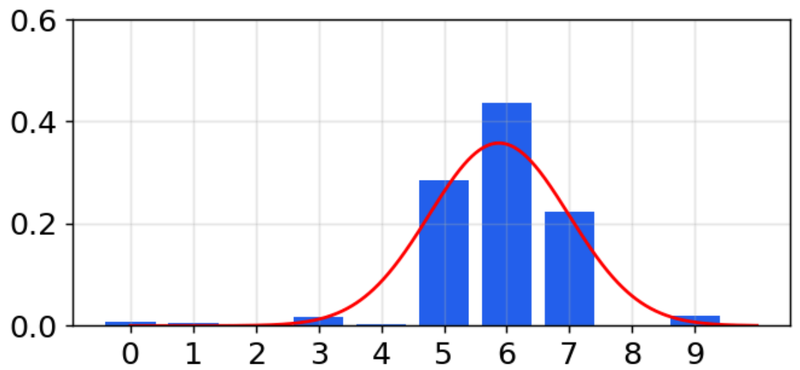
- 앞의 posterior 배열에서 평균과 분산을 계산해 Gaussian 곡선을 막대 위에 겹쳐 그림
- 평균 5.88, 분산 1.24 인 Gaussian 이 막대 모양과 잘 맞음
- 수백만 칸 분포도 두 숫자로 표현, update 는 곱셈 세 번과 나눗셈 두 번으로 충분
- **조건**: 평균 = sum(p x), 분산 = sum((x - mean)^2 p), x 는 칸 번호 0 ~ 9

```
xs = np.arange(0, 10, .01)

def mean_var(p):
    x = np.arange(len(p))
    mean = np.sum(p * x,dtype=float)
    var = np.sum((x - mean)**2 * p)
    return mean, var

mean, var = mean_var(posterior)
plots.bar_plot(posterior, ylim=(0, .6))
plt.plot(xs, gaussian(xs, mean, var, normed=False), c='r');
print('mean: %.2f' % mean, 'var: %.2f' % var)
```

```
mean: 5.88 var: 1.24
```



Bayes' Theorem

$$P(A \mid B) = \frac{P(B|A)\,P(A)}{P(B)} \qquad p(x_i \mid z) = \frac{p(z|x_i)\,p(x_i)}{p(z)}$$

항	이름	코드에서
p(x_i)	prior	측정 반영 전 믿음 `prior`
p(z | x_i)	likelihood	각 위치에서 그 측정이 나올 확률 `likelihood`
p(z)	evidence	위치와 무관한 측정 확률, `sum(belief)` 로 정규화
p(x_i | z)	posterior	측정 반영 후 믿음, `update()` 의 결과

> **핵심**: update() 는 Bayes 정리 그 자체, 조건부 확률 P(A | B) 는 B 가 일어났을 때 A 의 확률

Why Bayes Helps

$$p(A \mid B) = \frac{p(B|A)\,p(A)}{\int p(B|A_j)\,p(A_j)\,dA_j} \qquad p(x_i \mid Z) \propto p(Z \mid x_i)\,p(x_i)$$

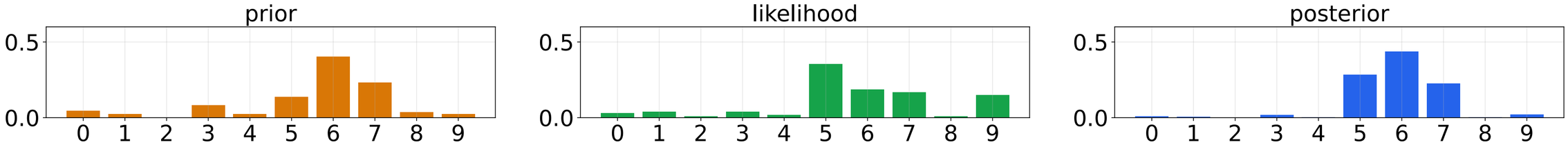
적분 분모

- 해석적으로 풀기 어려운 경우가 대부분
- 실제로는 정규화를 위한 합, 합으로 생각하면 쉬워짐

역방향 계산

- p(x | z) 는 직접 구하기 어려움
- p(z | x), 그 위치에서 그 측정이 나올 확률은 구하기 쉬움

예: 센서 값으로 비 올 확률 대신, 비가 올 때 그 센서 값이 나올 확률을 계산





Total Probability Theorem

- predict() 는 이동할 수 있는 모든 경우의 확률을 더해 각 위치의 새 확률을 계산
- 시각 t 에 위치 i 일 확률 = 이전 위치 j 확률 × j 에서 i 로 이동할 확률의 합
- 이 식이 전확률 정리, update 는 Bayes 정리이고 predict 는 전확률 정리
- **조건**: 위치 N 칸, 크기 kN 인 이동 확률 kernel, offset 만큼 이동

$$P(X_i^t) = \sum_j P(X_j^{t-1})\, P(x_i \mid x_j)$$

```
for i in range(N):
    for k in range (kN):
        index = (i + (width-k) - offset) % N
        result[i] += prob_dist[index] * kernel[k]
```

scipy.stats

- scipy.stats.norm(2, 3) 으로 평균 2, 표준편차 3 인 frozen 분포 객체를 만듦
- 1.5 에서의 pdf 가 Gaussian 식으로 직접 계산한 값과 같음, cdf(2) 는 0.5
- 같은 객체로 pdf, cdf, 분산, 표준편차, 평균을 반복 계산 가능
- **조건**: norm 은 표준편차 3 을, Gaussian 식 비교는 분산 3 x 3 = 9 를 사용

```python
from scipy.stats import norm
import filterpy.stats
print(norm(2, 3).pdf(1.5))
print(filterpy.stats.gaussian(x=1.5, mean=2, var=3*3))

n23 = norm(2, 3)
print('pdf of 1.5 is      %.4f' % n23.pdf(1.5))
print('pdf of 2 is        %.4f' % n23.pdf(2))
print(n23.cdf(2))   # P(x <= mean)
print('variance', n23.var(), 'std', n23.std(), 'mean', n23.mean())
```

```
0.13114657203397997
0.13114657203397995
pdf of 1.5 is      0.1311
pdf of 2 is        0.1330
0.5
variance 9.0 std 3.0 mean 2.0
```

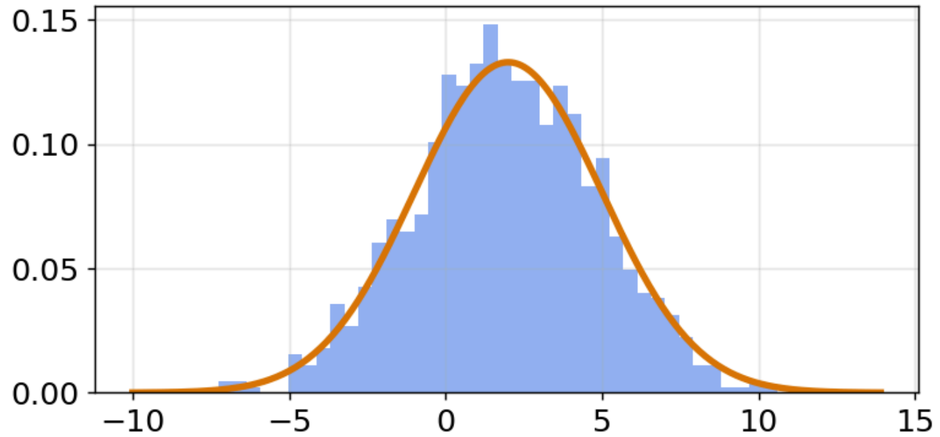
Sampling

- frozen 분포 norm(2, 3) 의 rvs() 로 그 분포를 따르는 표본 1,000 개를 생성
- 정규화한 히스토그램이 pdf 곡선과 거의 겹침
- **조건**: norm(2, 3), 표본 1,000 개, 40 칸 히스토그램 (density=True)

```
n23 = norm(2, 3)
samples = n23.rvs(size=1000)
print(samples[:5])

xs = np.arange(-10, 14, .1)
plt.hist(samples, bins=40, density=True, alpha=.5);
plt.plot(xs, n23.pdf(xs), lw=3)
```

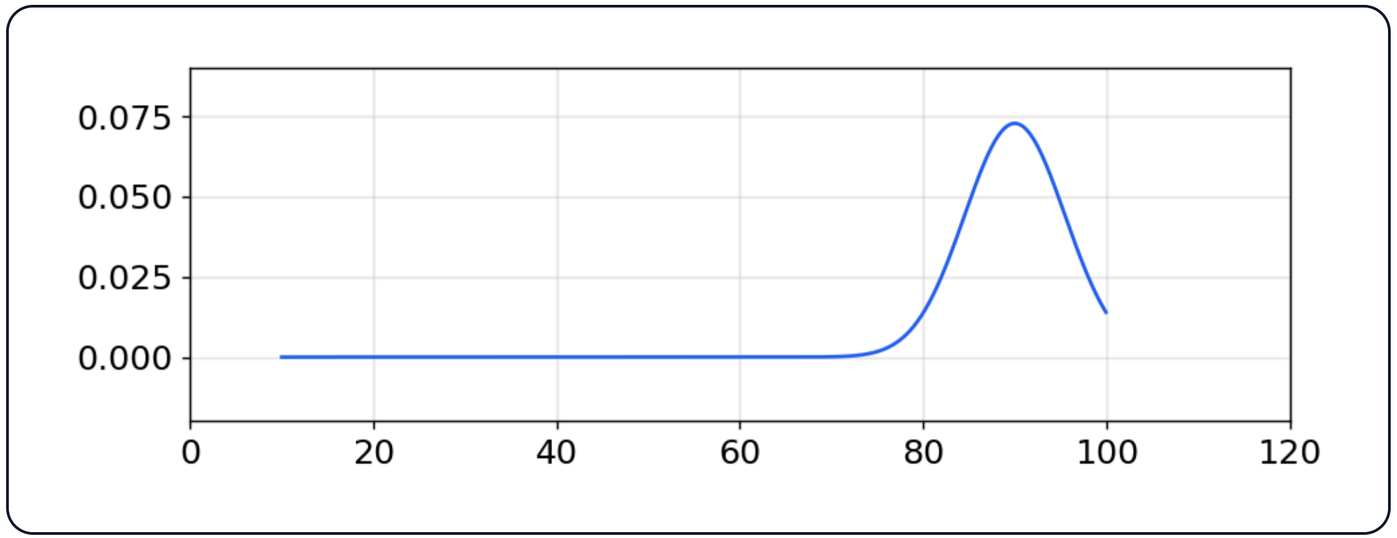
```
[ 7.072  0.602  2.098  3.223 -0.367]
```



Limitations of Gaussians

- 시험 점수를 평균 90, 분산 30 인 Gaussian 으로 모델링하고 10 ~ 100 점 구간을 그림
- 100 점 위쪽에도 확률이 남아 0 ~ 100 안의 넓이가 1 이 되지 못함
- 중심 극한 정리는 특정 조건에서만 성립, 실제 분포는 꼬리가 두꺼운 경우가 많음
- **조건**: 점수 범위 0 ~ 100, 평균 90, 분산 30

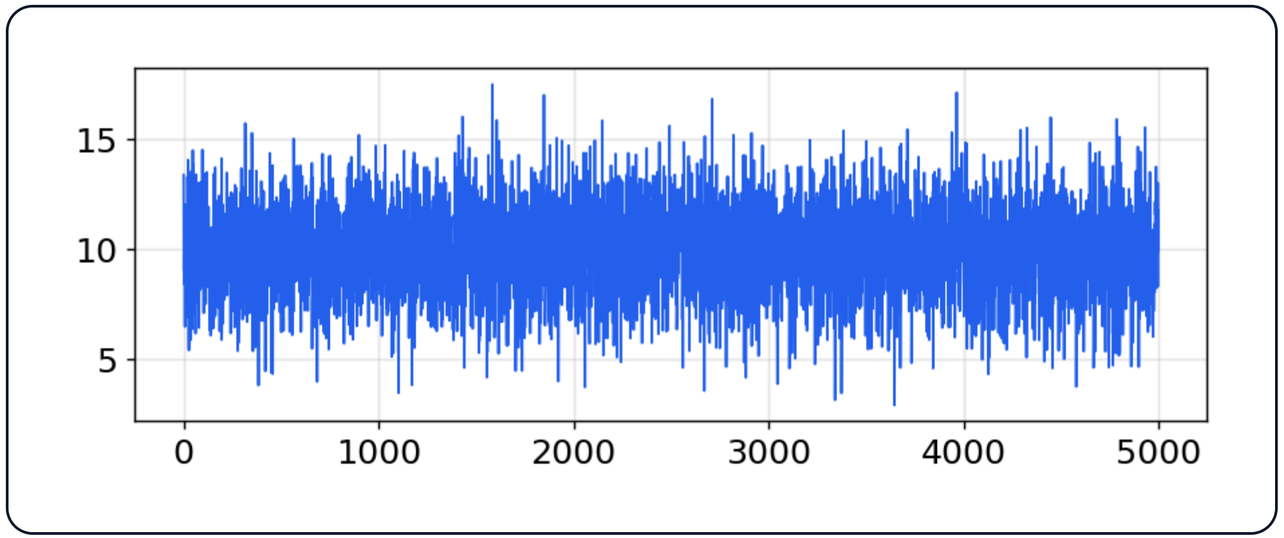
```
xs = np.arange(10, 100, 0.05)
ys = [gaussian(x, 90, 30) for x in xs]
plt.plot(xs, ys, label='var=0.2')
plt.xlim(0, 120);
plt.ylim(-0.02, 0.09);
```



Gaussian Sensor Noise

- 일정한 신호 10 에 표준편차 2 인 white noise 가 더해지는 센서를 randn() 으로 모의
- 측정값은 10 을 중심으로 분포, 68% 는 10 ± 2, 99% 는 10 ± 6 안
- 실제 센서 오차는 Gaussian 이 아닌 경우가 많음, Kalman filter 수학은 이상화된 모델
- **조건**: 신호 10, 잡음 표준편차 2, 측정 5000 회

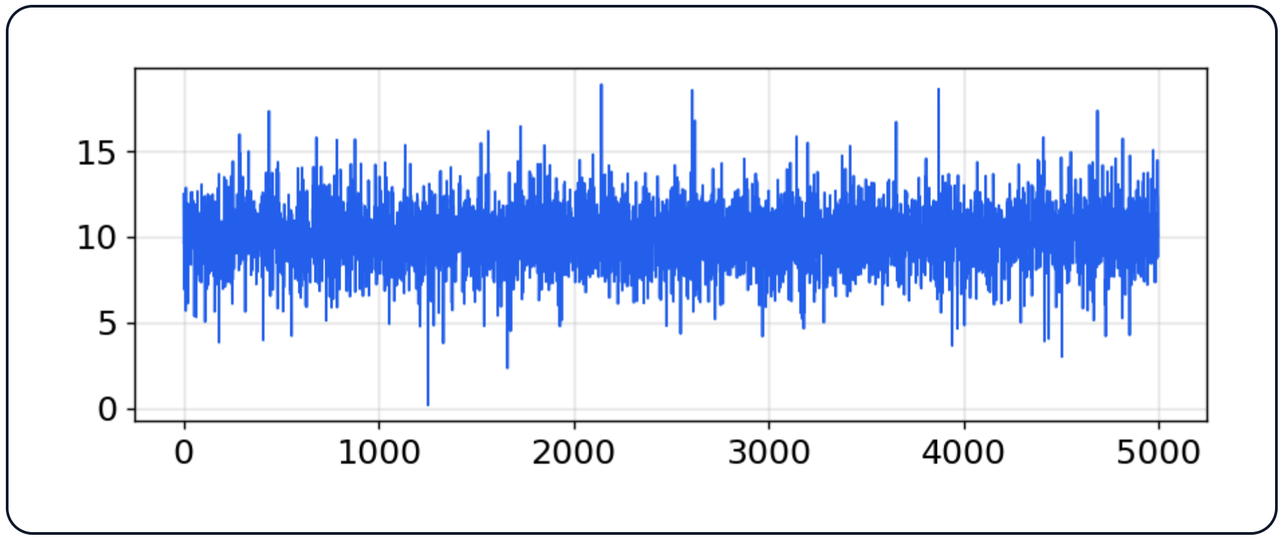
```
from numpy.random import randn
def sense():
    return 10 + randn()*2
zs = [sense() for i in range(5000)]
plt.plot(zs, lw=1);
```



Student's t Noise

- 같은 센서의 잡음을 자유도 7 인 Student t 분포에서 뽑아 5000 회 측정
- 정규분포와 비슷하지만 3 표준편차(7 ~ 13)를 훨씬 넘는 이상값이 자주 발생
- 실제 센서의 정확한 모델은 아니지만 현실적인 잡음으로 필터를 시험할 때 유용
- **조건**: 신호 10, 배율 2, 자유도 df = 7, 측정 5000 회

```python
import random, math
def rand_student_t(df, mu=0, std=1):
    x = random.gauss(0, std)
    y = 2.0*random.gammavariate(0.5*df, 2.0)
    return x / (math.sqrt(y / df)) + mu
def sense_t():
    return 10 + rand_student_t(7)*2
zs = [sense_t() for i in range(5000)]
plt.plot(zs, lw=1);
```



Skew and Kurtosis

- 정규분포 표본 10 개와 300,000 개의 통계를 scipy.stats.describe 로 비교
- 작은 표본은 평균, 분산이 이론값 0, 1 과 다르고 skew, kurtosis 도 0 에서 크게 벗어남
- skew 는 평균 기준 비대칭 정도, kurtosis 는 꼬리 두께, 큰 표본은 모두 0 에 가까움
- **조건**: 표준 정규분포 N(0, 1), 표본 수 10 과 300,000

```
1 import scipy.stats
2 print(scipy.stats.describe(np.random.randn(10)))
3 print()
4 print(scipy.stats.describe(np.random.randn(300000)))
```

```
DescribeResult(nobs=10, minmax=
(-1.7547243063454208,
1.690525703800356),
mean=0.07406091535945156,
variance=0.9205510622838874,
skewness=-0.22151012624778627,
kurtosis=-0.1540730087546116)
```

> **예**: 우주 임무에서는 이론상 3 표준편차 대신 실험으로 5 ~ 6 표준편차를 써야 했던 사례

Product of Gaussians

$$p(x \mid z) \propto p(z \mid x)\, p(x) \propto \exp\left[-\frac{(z-x)^2}{2\sigma_z^2}\right] \exp\left[-\frac{(x-\bar{\mu})^2}{2\bar{\sigma}^2}\right]$$

$$\propto \exp\left[-\frac{1}{2}\frac{\left(x-\frac{\sigma_z^2\bar{\mu}+\bar{\sigma}^2 z}{\bar{\sigma}^2+\sigma_z^2}\right)^2}{\frac{\sigma_z^2\bar{\sigma}^2}{\bar{\sigma}^2+\sigma_z^2}}\right]$$

$$\mu_{posterior} = \frac{\sigma_z^2\bar{\mu}+\bar{\sigma}^2 z}{\bar{\sigma}^2+\sigma_z^2} \qquad \sigma^2_{posterior} = \frac{\sigma_z^2\bar{\sigma}^2}{\bar{\sigma}^2+\sigma_z^2}$$

> **유도**: 상수 항을 버리고 x 에 대해 완전제곱으로 정리, 정규화는 Bayes 의 p(z) 가 담당

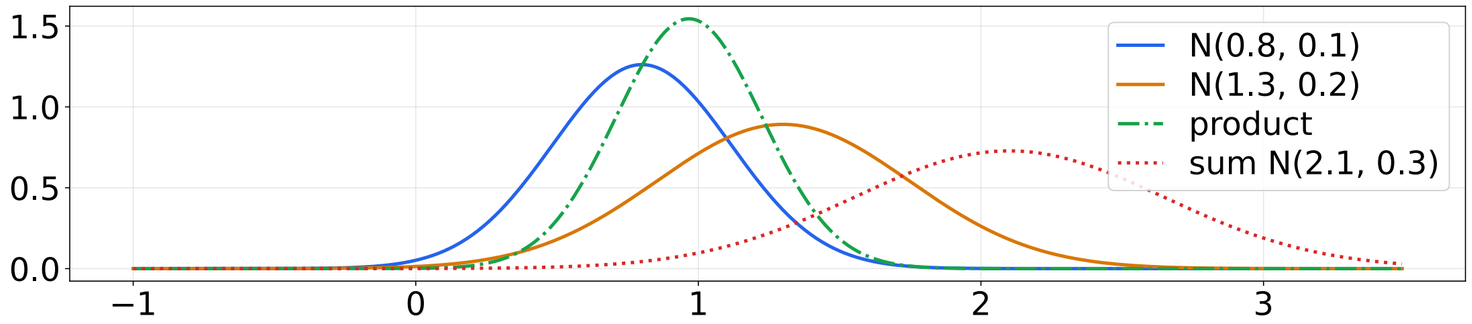
Sum of Gaussians

$$p(x) = \int_{-\infty}^{\infty} f_p(x - z) f_z(z)\, dz$$

$$p(x) = \frac{1}{\sqrt{2\pi}\sqrt{\sigma_p^2+\sigma_z^2}} \exp\left[-\frac{(x-(\mu_p+\mu_z))^2}{2(\sigma_z^2+\sigma_p^2)}\right]$$

$$\mu_x = \mu_p + \mu_z \qquad \sigma_x^2 = \sigma_z^2 + \sigma_p^2$$

> **유도**: 두 확률 변수의 합의 밀도는 합성곱, 적분 안의 정규분포 넓이가 1 이 되어 정리



Summary

- **Gaussian**: 연속 확률 분포, 평균 mu 와 분산 sigma^2 두 매개변수로 완전히 표현
- **mu**: 모든 가능한 값의 평균
- **sigma^2**: 측정이 평균에서 얼마나 퍼지는지
- **sigma**: 분산의 제곱근, 데이터와 같은 단위
- **한계**: 자연의 많은 현상이 정규분포에 가깝지만 수학이 완벽하지는 않음
- **Bayes**: p(x | z) 는 어렵고 p(z | x) 는 쉬움, Bayes 로 앞의 것을 계산

> **결론**: Gaussian 과 Bayes 정리의 결합이 필터 구성의 기반